

Deep Learning for teens

By Hamid Sadeghi

Table of contents

- ▶ Session1
- ▶ Session2
- ▶ Session3
- ▶ Session4
- ▶ Session5
- ▶ Session6
- ▶ Session7
- ▶ Session8
- ▶ Session9
- ▶ Session10
- ▶ Session11
- ▶ Session12
- ▶ Session13
- ▶ Session14
- ▶ Session15

Session1

- ▶ What is a neuron: smallest part of a NN
- ▶ Layers in NN: input layer, hidden layer and output layer
- ▶ Neural network explanation, split :train, test and validation
- ▶ Activation function: if a neuron fires or not
- ▶ Loss : measure how far predictions are from the true labels
- ▶ Forward Pass: going from input to output
 - ▶ Backward error propagation: Calculating the loss gradient at the output layer
 - ▶ Propagating that error backward through each layer using the chain rule
 - ▶ Updating weights using gradient descent
- ▶ Gradient descent : Gradient descent is an optimization algorithm that updates weights by moving in the direction that reduces loss.
- ▶ decision boundary: Imagine you have two groups of points: **red dots** and **blue dots**
 - ▶ A decision boundary is the line (or curve) the model draws to separate the red from the blue.

Session 2

- ▶ ANN vs NN: NN general term for any model made of connected artificial neurons.
 - ▶ the basic feedforward NN (input → hidden → output), used for regression/classification.
 - ▶ Every ANN is an NN, but not every NN is an ANN.
- ▶ Dense Layer(fully connected layer):**every neuron is connected to every neuron in the next layer.**
- ▶ Activation function:
- ▶ Dense Layer + Activation Function Choices
- ▶ Optimizer:In deep learning, an optimizer is the algorithm that updates the weights of your network to minimize the loss function
- ▶ Batch gradient decent, Stochastic gradient decent vs Mini batch gradient decent
- ▶ Forward Pass Breakdown
- ▶ Loss
- ▶ Metric:it's just a way to measure how well your model is performing
- ▶ batch_size

Session3

- ▶ Batch gradient decent, Stochastic gradient decent vs Mini batch gradient decent
- ▶ Dropout layer(.2): randomly turns off 20% of the neurons during training, leaving 80% active.
- ▶ Flatten layer: Flatten turns multi-dimensional input into a 1-D vector.
- ▶ main types of layers in a CNN: input Layer, Convolution Layer, Activation Layer, Pooling Layer, Fully Connected Layer, Output Layer
- ▶ Convolutional Neural Network (CNN) architecture diagram
- ▶ ANN vs CNN: Why Convolutions Beat Flattening for Image Classification

Session4

- ▶ Transfer learning:
 - ▶ We freeze all the layers except the last one.
 - ▶ When we do that, the model's weights don't change during training.
 - ▶ `base_model.trainable = False`
- ▶ Transfer Learning vs Fine-Tuning
- ▶ Fine-Tuning
 - ▶ Start from pre-trained weights.
 - ▶ Unfreeze some or all layers and retrain them with your dataset.
 - ▶ Best for larger datasets or when your task is quite different from the original training data.

Session5

- ▶ Data Augmentation
- ▶ Reasons
 - ▶ Increase the amount of training data
 - ▶ Improve performance on unseen images
 - ▶ Reduce overfitting on small datasets

Session6

- ▶ RNN: Mainly use for NLP task
- ▶ Real life use case of RNN
- ▶ Why we can't use simple neural network for sequences problem
 - ▶ ANN needs fixed input size
 - ▶ ANN cannot understand word order
- ▶ Vanilla RNN: has only short term memory
- ▶ Vanishing memory in RNN
- ▶ RNN Input-Output Types
- ▶ LSTM RNN
- ▶ GRU RNN

Session7

- ▶ **Vanilla RNN** → basic, simple memory
- ▶ **LSTM** → long-term memory, avoids vanishing gradient
- ▶ **GRU** → simpler than LSTM, similar performance
- ▶ **Bidirectional RNN** → reads sequence forward + backward
- ▶ **Encoder-Decoder RNN** → used for translation, seq2seq
 - ▶ The encoder reads the whole input and turns it into one meaning.
 - ▶ The decoder takes that meaning and produces a new sequence from it.
- ▶ **Stacked RNN** → multiple RNN layers on top of each other

Session8

- ▶ RNN doesn't care if the elements are words or numbers
- ▶ Embeddings fix the meaning problem
 - ▶ Embeddings turn each word into a **meaningful vector**.
 - ▶ Why RNN cannot create embeddings by itself
 - ▶ Embedding capture similarities between words
- ▶ Embeddings
 - ▶ Supervised learning of Embeddings
 - ▶ Unsupervised learning Embeddings
 - ▶ Word2Vec
 - ▶ is a method that learns word embeddings – meaning it learns a vector for each word such that:
 - ▶ It doesn't care about grammar, rules, or linguistics.
 - ▶ It only cares about patterns of co-occurrence.

Session9

- ▶ Embedding
 - ▶ Supervised learning:
 - ▶ (no major embedding algorithm)
 - ▶ Self-supervised learning:
 - ▶ Word2Vec
 - ▶ FastText
 - ▶ Unsupervised learning:
 - ▶ GloVe

Session10

- ▶ ☆ Why ordinary word embeddings don't work well for sentences
 - ▶ One meaning only
 - ▶ Each word gets one fixed vector, so it can't change meaning based on the sentence.
 - ▶ No sense of order
 - ▶ They don't know which word comes first or last, so the structure of the sentence is lost.
 - ▶ They forget long sentences
 - ▶ They can't keep track of how early words relate to later words.
- ▶ What is BERT
 - ▶ Think of BERT as a model that **reads text like a human** — not just left-to-right, not just right-to-left, but **both directions at the same time**.
- ▶ BERT can make contextualized embedding means
 - ▶ A contextualized embedding is a word vector whose meaning changes depending on the sentence around it.
 - ▶ In older models (Word2Vec, GloVe), each word had one fixed vector, no matter where it appeared.

Session11

- ▶ BERT
 - ▶ Think of BERT as a model that **reads text like a human** — not just left-to-right, not just right-to-left, but **both directions at the same time**.
- ▶ Transformers got two components
 - ▶ Encoder
 - ▶ Converts whole sentence to contextual embedding
 - ▶ Decoder
 - ▶ We give the embedding to decoder to do something for us like text prediction, translating sentence
- ▶ BERT has only encoder part
- ▶ GPT has only decoder part

Session12

- ▶ Used to make big model the smaller one
- ▶ Then u can deploy them on edge devices like micro controller
- ▶ There two way to perform quantization in tensorflow
 - ▶ Post training quantization
 - ▶ Down side is accuracy might get suffered . Better approach is using
 - ▶ Quantization aware training
 - ▶ Quantization aware training
 - ▶ You take the model, then u apply quantized function on it, then u take quantized model in tensorflow
 - ▶ Then u do training again (like transfer learning) for fewer epochs
 - ▶ U get fine tuned quantized model
 - ▶ Then u use tf lite (if u want to deploy models on edge devices , u need to use tf lite)

Session13

- ▶ TF Input Pipeline
 - ▶ A TensorFlow Input Pipeline is just a way to get your data ready for your model. It loads your data step-by-step so your model never has to wait.
- ▶ Optimize tensorflow pipeline
 - ▶ Step 1: Cache the Dataset
 - ▶ `dataset = dataset.cache()` # This will cache the dataset in memory after the first load
 - ▶ Step 2: Prefetch the Data
 - ▶ `dataset = dataset.prefetch(buffer_size=tf.data.AUTOTUNE)` # This will allow later elements to be prepared while the model is training on current elements

Session14

Tensor flow vs pytorch

Pytorch is by facebook

Tesorflow is by google

Topic	<u>PyTorch</u>	<u>TensorFlow</u>
Coding Style	Feels like normal Python. Easy to write.	More structured. Keras makes it simple.
How It Runs	Runs line-by-line. Easy to debug.	Can run line-by-line or as a fast graph.
Who Uses It	Researchers, universities.	Companies, production teams.
Deployment	Okay for deployment.	Very strong (mobile, web, servers).
Learning Curve	Very easy to start.	Easy with Keras, more tools overall.

Session15

- ▶ Yolo
 - ▶ YOLO does **real-time object detection** — it finds and labels objects in an image in one single pass.
- ▶ Ml-flow
 - ▶ MLflow logs metrics like accuracy, loss, MAE, MSE, so you can compare different training runs.